

Documentación exhaustiva — Proyecto CuBirds

(Tercera Entrega)

Índice

- Resumen rápido
- Reglas del juego mapeadas al código
- Estructura del repositorio (archivos y paquetes)
- Clases y API detallada (métodos públicos, comportamiento, ejemplos)
- Uso: compilar, ejecutar y jugar (ejemplos)
- Buenas prácticas y pruebas
- Entrega y versión

Resumen rápido

- Implementación en Java del juego CuBirds siguiendo el enunciado de la asignatura.
- Objetivo del juego (resumido): ser el primer jugador en conseguir 7 especies diferentes en su zona de juego (colección). Existe además la regla auxiliar de bandadas (bandada pequeña/grande) que condiciona cuántas cartas descartar para puntuar una especie.

Reglas del juego (mapa conceptual → comportamiento en el código)

- Preparación (clases implicadas):
 - `DeckOfCards`: crea y baraja el mazo.
 - `Table`: coloca 4 filas (vallas) con 3 cartas cada una, asegurando que inicialmente no haya especies repetidas en una misma fila.
 - `Game` crea la baraja, la mesa y, en su constructor, inicializa la mesa con 3 cartas por fila.
 - `Game.repartirCartas()`: reparte 8 cartas a cada jugador.
- Turno de jugador (flujo principal en `Game.executePlayerTurn`):
 1. Mostrar mano y mesa (`IU` + llamadas `toString()` de `Player` y `Table`).
 2. Elegir especie jugable (`Player.getPlayableSpecies()` y `IU.chooseBirdType(...)`).
 3. Elegir fila y lado (`IU.chooseRow()`, `IU.chooseSide()`).
 4. Colocar cartas: `Table.placeCardsOnRow(cardsToPlay, rowIndex, placeLeft)`.
 5. Recoger cartas rodeadas: `placeCardsOnRow` devuelve las cartas capturadas que se añaden a la mano del jugador.
 6. Si tras la captura la fila quedó con una única especie,
`Table.ensureRowHasTwoSpecies(rowIndex, deck, discardedCards)` rellena desde la baraja (o desde descartes si la baraja se queda vacía) hasta obtener otra especie.
 7. Opcional: el jugador puede bajar una especie a su zona de juego
(`Game.handleCollectionChoice`) si tiene al menos `smallFlock` cartas de esa especie en mano; entonces se incrementa su contador de especie (zona de juego) y se envían al `DiscardedCards` todas las cartas de esa especie que tenga en la mano.
 8. Comprobación de victoria: si `player.getCollectedSpeciesCount() >= 7`, termina la partida.

- Fin de ronda por mano vacía (`Game.handleEmptyHand`):
 - Si el jugador activo se queda sin cartas, los demás se descartan por completo (sus manos van a `DiscardedCards`).
 - Se vuelcan los descartes a la baraja (`DiscardedCards.moveAllToDeck(deck)`) y se baraja; se reparte nuevamente 8 cartas por jugador si es posible.
 - Si no hay suficientes cartas para repartir 8 por jugador, el juego termina y gana el jugador con más cartas en su colección (`Game.getWinnerByCollection()`).

Estructura del repositorio

Raíz del módulo de código: `Proyecto/` (contiene `pom.xml` y `src/`).

- Código principal Java: `src/main/java/gal/uvigo/esei/aed1/cubirds/`
 - `core` — lógica y modelo del juego.
 - `iu` — interfaz de usuario (consola) y `Main`.
- Código auxiliar (TADs) añadido en `src/tads` (añadido al classpath por `pom.xml`): implementaciones simples de `List`, `Queue`, `Stack` usadas por el proyecto.

Archivos importantes

- `Proyecto/pom.xml` — configuración Maven del proyecto.
- `src/main/java/gal/uvigo/esei/aed1/cubirds/iu/Main.java` — punto de entrada (si lo usas desde IDE o `mvn exec`).
- `src/main/java/gal/uvigo/esei/aed1/cubirds/iu/IU.java` — interacción por consola (leer números, strings, mostrar menús).
- `src/main/java/gal/uvigo/esei/aed1/cubirds/core/*` — clases del juego (listadas y documentadas más abajo).

Nota rápida sobre la versión actual del código:

- `Card` es un `enum`, no una clase con instancias mutables.
- `Game` inicializa la mesa en el constructor llamando a `table.inicializarMesa(deck)`.

Clases y API detallada

La sección siguiente lista las clases clave del paquete `core` y describe sus métodos públicos (firma, comportamiento y ejemplos de uso).

1. `TypeBird` (enumeración)

- Descripción: enum que representa las especies de pájaros del juego. Cada constante incluye parámetros (ej.: número de cartas en la baraja, umbral de bandada pequeña/grande si aplica).
- Uso típico:
 - `TypeBird.SOME_SPECIES` — acceder a la especie.
 - `TypeBird.values()` — iterar todas las especies.

2. `Card`

- Descripción: enumeración de cartas del mazo. Cada constante representa una carta concreta y contiene `TypeBird typeBird`, `int smallFlock` (bandada pequeña), `int largeFlock` (bandada grande) y representación textual.
- Métodos públicos relevantes (ejemplo de firma):
 - `TypeBird getTypeBird()` — devuelve la especie de la carta.
 - `int getSmallFlock()` — devuelve el tamaño de bandada pequeña de la carta.
 - `String toString()` — representación amigable para mostrar en la IU.

3. DeckOfCards

- Descripción: estructura que representa la baraja. Opera como deque (robar del frente, meter al final, barajar).
- Métodos públicos:
 - `DeckOfCards()` — crea todas las cartas en orden y las baraja.
 - `Card takeFirstCard()` — saca la carta superior del mazo. Lanza si está vacío (la lógica de juego revisa el montón de descartes antes de llamar si procede).
 - `void addLast(Card c)` — añade una carta al final de la baraja (se usa para devolver cartas no utilizadas al final).
 - `boolean isEmpty()` — indica si la baraja está vacía.
 - `int size()` — número de cartas en el mazo.
 - `void shuffle()` — mezcla el mazo.

4. DiscardedCards

- Descripción: acumulador de cartas descartadas. Implementa:
 - `void addCard(Card card)` — añade una carta al montón.
 - `void addCards(List<Card> cards)` — añade varias.
 - `boolean isEmpty()` — true si no hay descartes.
 - `int size()` — tamaño del montón.
 - `void moveAllToDeck(DeckOfCards deck)` — pasa todas las cartas descartadas al mazo y baraja el mazo.

5. Table

- Descripción: representa las 4 filas del tablero y operaciones sobre las filas.
- Atributos principales: `List<Card>[] filas` (4 listas).
- Métodos públicos y comportamiento:
 - `Table()` — constructor que crea las 4 listas vacías.
 - `void inicializarMesa(DeckOfCards deck)` — coloca 3 cartas por fila: extrae cartas del mazo y, si la carta causa duplicado de especie dentro de la fila, devuelve la carta al final del mazo hasta completar 3 especies distintas.
 - `int getRowCount()` — devuelve 4.
 - `List<Card> placeCardsOnRow(List<Card> cardsToPlay, int rowIndex, boolean placeLeft)` — coloca las cartas `cardsToPlay` en la fila `rowIndex`, en el lado izquierdo si `placeLeft==true` o derecho si es `false`.
 - Comportamiento: inserta las cartas (sin necesidad de invertir la lista de entrada). Busca si hay cartas del mismo tipo en la fila; si existe, captura las cartas entre las posiciones correspondientes y las devuelve en una lista `capturedCards`.
 - Devuelve `List<Card>` con las cartas capturadas; si no hay capturas, devuelve lista vacía.

- `void ensureRowHasTwoSpecies(int rowIndex, DeckOfCards deck, DiscardedCards discardedCards)` — cuando una fila queda con todas cartas de la misma especie, toma cartas del mazo hasta introducir otra especie. Si el mazo se vacía, vuelca descartes al mazo y continúa.
- `String toString()` — representación para mostrar la mesa por consola.

6. Player

- Descripción: modelo de jugador. Atributos públicos/claves: `String name`, `List<List<Card>> hand` (mano agrupada por especie), `int[] speciesCounters` (zona de juego: contador de cartas por especie).
- Métodos públicos:
 - `Player(String name)` — constructor.
 - `String getName()`.
 - `int getHandSize()` — total de cartas en la mano.
 - `int getCollectedSpeciesCount()` — número de especies distintas que el jugador ha bajado (contador > 0).
 - `int getCollectionSize()` — número total de cartas en la colección (sumatorio de counters).
 - `void incrementSpeciesCounter(TypeBird species)` — incrementa en 1 el contador de la especie en la zona de juego.
 - `List<TypeBird> getPlayableSpecies()` — devuelve lista de especies que el jugador puede jugar (una por cada grupo en la mano). Nota: los grupos vacíos no deberían existir; por eso no se comprueba `isEmpty()` en la implementación actual.
 - `int getHandCountForSpecies(TypeBird species)` — número de cartas de esa especie en mano.
 - `int getSmallFlockForSpecies(TypeBird species)` — devuelve el umbral de bandada pequeña asociado a la especie (se delega en la primera carta del grupo).
 - `void addCardToHand(Card card) / void addCardsToHand(List<Card> cards)` — añaden cartas a la mano agrupadas por especie.
 - `List<Card> takeCardsOfSpecies(TypeBird species)` — quita y devuelve el grupo completo de cartas de la mano para esa especie.
 - `List<Card> takeAllCards()` — vacía por completo la mano devolviendo todas las cartas (usado cuando un jugador se queda sin cartas y los demás deben descartarse).
 - `boolean hasNoCards()` — true si la mano está vacía.
 - `String toString()` — representación de la mano para mostrar por consola.

7. Game

- Descripción: controla el flujo completo de la partida.
- Atributos principales: `IU iu`, `DeckOfCards deck`, `Table table`, `Player[] players`, `DiscardedCards discardedCards`.
- Métodos relevantes:
 - `Game(IU iu)` — constructor: inicializa mazo, mesa y descartes.
 - `void inicializarJugadores()` — pide por consola el número de jugadores (2-5) y sus nombres, creando `Player` para cada uno.
 - `void repartirCartas()` — reparte 8 cartas a cada jugador.
 - `void mostrarEstadoInicial()` — imprime mesa y manos.
 - `TypeBird elegirTipo(Player player)` — obtiene especies jugables y pide elección (IU).

- `int elegirFila()` — pide fila al usuario (`IU.chooseRow`) — ahora muestra **Elige una fila (1-4):**.
- `boolean elegirLado()` — pide lado (izquierda/derecha).
- `void executePlayerTurn(Player player)` — ejecuta todo el turno: mostrar mano/mesa, elegir especie, colocar cartas, añadir cartas capturadas a la mano, rellenar fila si procede, ofrecer opción de bajar especie a zona de juego, etc.
- `void play()` — bucle principal del juego: inicializa jugadores, reparte, ciclo de turnos, detecta victoria o fin de reparto por falta de cartas.
- `boolean handleEmptyHand(Player currentPlayer)` — si `currentPlayer` se queda sin cartas, fuerza a los otros a descartarse (añadiendo sus cartas a `discardedCards`), vuelca descartes en `deck` y reparte; devuelve `true` si no es posible repartir y la partida debe finalizar.
- `Player getWinnerByCollection()` — determina ganador por mayor colección en caso de final por falta de reparto.

8. IU (Interfaz de Usuario)

- Descripción: utilidades de lectura y display por consola usando `Scanner`.
- Métodos relevantes:
 - `int readNumber(String msg)` — lee un entero; reintenta hasta entrada válida.
 - `String readString(String msg)` — lee una línea.
 - `void displayMessage(String msg)` — imprime por consola.
 - `TypeBird chooseBirdType(List<TypeBird> availableTypes)` — muestra lista numerada y devuelve elección.
 - `int chooseRow(int rowCount)` — muestra filas disponibles y pide número; devuelve índice 0-based. Mensaje ahora **Elige una fila (1-4):**.
 - `boolean chooseSide()` — pide 1 izquierda, 2 derecha.
 - `boolean chooseYesNo(String message)` — pide 1 si, 2 no.

Compilación y Ejecución — Guía Completa

Requisitos del sistema

- **Java 17+** (OpenJDK 17, Eclipse Adoptium, u otra distribución)
- **Maven 3.6+** (recomendado 3.9.x)
- Ubicación: desde la carpeta `Proyecto/` (donde está `pom.xml`)

Compilación paso a paso

Paso 1: Verifica Maven e instalación Java

```
# Verifica Maven
mvn -version

# Verifica Java
java -version
javac -version
```

Paso 2: Limpia (opcional pero recomendado) Esto elimina compilaciones previas y la carpeta `target/`:

```
mvn clean
```

Paso 3: Compila y empaqueta en JAR Desde la carpeta **Proyecto/**:

```
mvn package
```

Salida esperada (al final):

```
[INFO] Building jar: ...\\target\\cubirds_PrimerEntrega-1.0-SNAPSHOT.jar  
[INFO] BUILD SUCCESS
```

Si quieres omitir pruebas (en caso de existir):

```
mvn package -DskipTests
```

Paso 4: Verifica la generación del JAR El archivo estará en:

```
target/cubirds_xxxx-x.x.x.jar
```

Verifica tamaño y existencia:

```
ls -l target/*.jar
```

Ejecución del JAR compilado

Opción A: Ejecución directa con java -jar (si pom.xml define Main-Class)

```
java -jar target/cubirds_PrimerEntrega-1.0-SNAPSHOT.jar
```

Opción B: Ejecución especificando la clase principal (recomendado)

```
java -cp target/cubirds_PrimerEntrega-1.0-SNAPSHOT.jar  
gal.uvigo.es.ei.aed1.cubirds.iu.Main
```

Opción C: Ejecutar directamente con Maven (sin generar JAR) Desde **Proyecto/**:

```
mvn exec:java -Dexec.mainClass="gal.uvigo.esei.aed1.cubirds.iu.Main"
```

Solución de problemas

Problema	Solución
mvn: command not found	Instala Maven o añádelo a PATH
java: command not found	Instala Java 17+ o añádelo a PATH
BUILD FAILURE: "No compiler is provided"	Asegúrate de tener JDK (no JRE) instalado
JAR no se ejecuta	Verifica que la clase <code>Main</code> existe en <code>gal.uvigo.esei.aed1.cubirds.iu.Main</code>
OutOfMemoryException en JAR	Incrementa memoria: <code>java -Xmx512m -cp ...</code>

Ejemplos y fragmentos de uso

Compilar con Maven desde la carpeta **Proyecto**:

```
mvn clean package
```

Ejecutar el juego (opción recomendada):

```
mvn exec:java -Dexec.mainClass="gal.uvigo.esei.aed1.cubirds.iu.Main"
```

O con el JAR:

```
java -cp target/cubirds_PrimerEntrega-1.0-SNAPSHOT.jar  
gal.uvigo.esei.aed1.cubirds.iu.Main
```

Flujo de juego en la consola (ejemplo simplificado):

1. Al iniciar: se pide número de jugadores (2-5) y nombres.
2. Se muestra la mesa (4 filas con 3 cartas) y las manos de cada jugador.
3. Turno 1: el primer jugador ve su mano y se le pide:
 - elegir especie jugable (p. ej. 1. **Gaviota**)
 - elegir fila (**Elige una fila (1-4):**)
 - elegir lado (1 izquierda / 2 derecha)
4. Se colocan las cartas; si captura cartas, estas se añaden a la mano.
5. Si la fila quedó con una única especie, la fila se rellena hasta que aparezca otra especie (robando del mazo o vaciando descartes en el mazo si es necesario).

6. Tras la jugada, se pregunta si desea bajar una especie a su colección (bandada pequeña). Si lo hace y tiene suficientes cartas, se incrementa el contador y las cartas se descartan en `DiscardedCards`.
7. Se comprueba victoria y se pasa turno.

Notas de sintaxis y estilo para desarrolladores

- Paquete raíz: `gal.uvigo.esei.aed1.cubirds` — seguir convención de paquetes para nuevas clases.
- Evitar cambiar las firmas públicas de métodos sin actualizar los usos en `Game` y `Table`.
- Si se añaden tests, colocar fuentes de test en `src/test/java` y configurar `pom.xml` si hace falta librerías de test (JUnit 5 por ejemplo).

Pruebas recomendadas (unitarias/manuales)

- Tests unitarios sugeridos:
 - `DeckOfCardsTest`: mezclar, tamaño, `takeFirstCard()` comportamiento cuando está vacío (esperar control externo), `shuffle()`.
 - `TableTest`: `inicializarMesa()` (cada fila inicia con 3 cartas distintas), `placeCardsOnRow()` devuelve capturas correctas para varios escenarios, `ensureRowHasTwoSpecies()` rellena correctamente consumiendo descartes si el mazo se vacía.
 - `PlayerTest`: agrupar cartas por especie, `takeCardsOfSpecies()` elimina correctamente y actualiza grupos.
 - `GameIntegrationTest`: simulación de varias rondas hasta victoria o fin de reparto.

Entrega y versión

- Fecha de la tercera entrega: 19 de mayo de 2026 (según enunciado).
- La rama `terceraEntrega` se ha fusionado sobre `main` en este repositorio para preparar la entrega final.

Ficheros relevantes con localización

- Código del juego: `Proyecto/src/main/java/gal/uvigo/esei/aed1/cubirds/core/` (clases `Card.java`, `DeckOfCards.java`, `DiscardedCards.java`, `Table.java`, `Player.java`, `Game.java`, `TypeBird.java`)
 - Interfaz: `Proyecto/src/main/java/gal/uvigo/esei/aed1/cubirds/iu/IU.java` y `Main.java`.
-